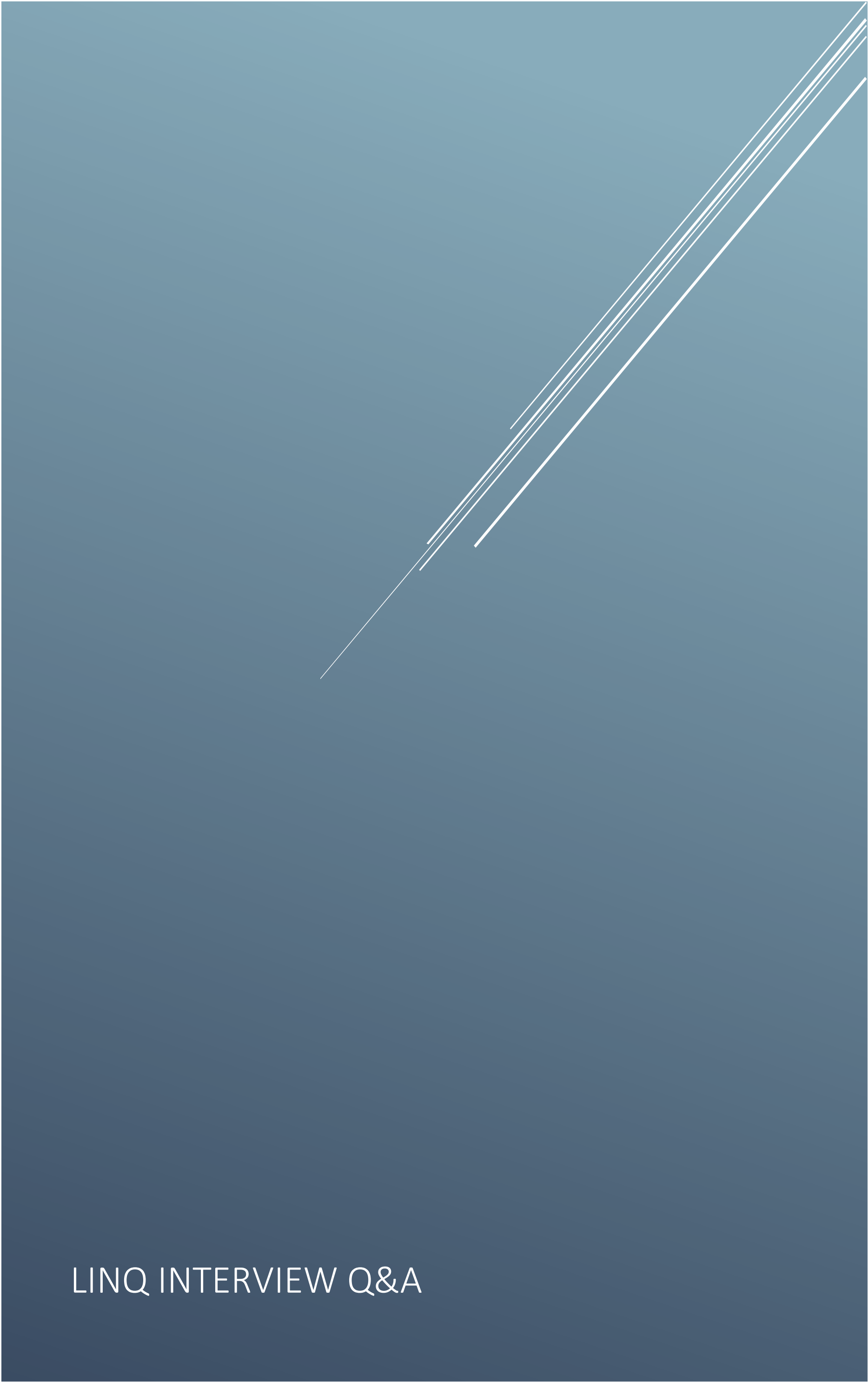


An abstract graphic featuring a blue gradient background that transitions from a lighter blue at the top to a darker blue at the bottom. In the upper right quadrant, there are several thin, white, parallel lines that originate from a single point and fan out towards the top right corner, creating a sense of motion or a stylized 'L' shape.

LINQ INTERVIEW Q&A

What is LINQ?

It enables you to query the data from the various data sources like SQL databases, XML documents, ADO.NET Datasets, Web services and any other objects such as Collections, Generics etc. by using a **SQL Query like syntax** with .NET framework languages like C# and VB.NET.

Why LINQ

LINQ has full **type checking** at compile-time and **IntelliSense** support in Visual Studio, since it used the .NET framework languages like C# and VB.NET. This powerful feature helps you to avoid run-time errors.

LINQ also provides a **uniform programming model** (i.e. common query syntax) to query various data sources. Hence you don't need to learn the different ways to query different data sources.

Which namespace is necessary to use LINQ?

System.Linq namespace is necessary for writing LINQ queries and to implement it.

What are advantages of LINQ?

There are following advantages of using LINQ:

1. It provides a uniform programming model (i.e. common query syntax) to query data sources (like SQL databases, XML documents, ADO.NET Datasets, Various Web services and any other objects such as Collections, Generics etc.)
2. It has full **type checking** at compile-time and **IntelliSense** support in Visual Studio. This powerful feature helps you to avoid run-time errors.
3. It supports various powerful features like filtering, ordering and grouping with minimum code.
4. **Its Query can be reused.**
5. It also allows debugging through .NET debugger.

What are disadvantages of LINQ?

There are following disadvantages of using LINQ:

1. LINQ is not good to write complex queries like SQL.
2. **LINQ doesn't take the full advantage of SQL features like cached execution plan for stored procedure.**
3. **Performance is degraded if you don't write the LINQ query correctly.**
4. If you have done some changes in your query, you have to recompile it and redeploy its dll to the server.

What is anonymous method?

An anonymous method is an inline unnamed method in the code. It is created using the **delegate keyword** and doesn't have its name and **return type**.

In this way, an anonymous method has no name, optional parameters and return type; it has only body. An anonymous method behaves like a regular method and allows you to write inline code in place of regular method.

Eg –

```
class Program
{
    //delegate for representing anonymous method
    delegate int del(int x, int y);
    static void Main(string[] args)
    {
        //anonymous method using delegate keyword
        del d1 = delegate(int x, int y) { return x * y; };
        int z1 = d1(2, 3);
        Console.WriteLine(z1);
    }
}
```

1. **A variable, declared outside the anonymous method can be accessed inside the anonymous method.**
2. A variable, declared inside the anonymous method can't be accessed outside the anonymous method.
3. **We use anonymous method in event handling.**
4. **An anonymous method, declared without parenthesis can be assigned to a delegate with any signature.**
5. **Unsafe code can't be accessed within an anonymous method.**
6. An anonymous method can't access the ref or out parameters of an outer scope.

What is lambda expression?

lambda expression is a more concise syntax of anonymous method. It is just a new way to write anonymous method. At compile time all the lambda expressions are converted into anonymous methods according to lambda expression conversion rules. The left side of the lambda operator "=>" represents the arguments of anonymous method and the right side represents the method body.

Eg-

Lambda expression Syntax (Input-parameters)

=> expression-or-statement-block

What is Extension method?

An extension method is a static method of a static class that can be invoked like as an instance method syntax. Extension methods are used to **add new behaviors to an existing type** without altering.

In extension method "**this**" keyword is used with the first parameter and the first parameter will be of **type** that is extended by extension method. Other parameters of extensions types can be of any types (data type).

Eg-

```
//defining extension method
public static class MyExtensions
{
    public static int WordCount(this String str)
    {
        return str.Split(new char[] { ' ', '.', ',' }).Length;
    }
}
class Program
{
    public static void Main()
    {
        string s = "Dot Net Tricks Extension Method Example";
        //calling extension method
        int i = s.WordCount();
        Console.WriteLine(i);
    }
}
//output:
//6
```

What is difference among Single, SingleOrDefault, First and FirstOrDefault?

Single - It returns a single specific element from a collection of elements if element match found. An exception is thrown, if none or more than one match found for that element in the collection.

SingleOrDefault - It returns a single specific element from a collection of elements if element match found. An exception is thrown, if more than one match found for that element in the collection. A default value is returned, if no match is found for that element in the collection.

First - It returns first specific element from a collection of elements if one or more than one match found for that element. An exception is thrown, if no match is found for that element in the collection.

FirstOrDefault - It returns first specific element from a collection of elements if one or more than one match found for that element. A default value is returned, if no match is found for that element in the collection.

When to use Single, SingleOrDefault, First and FirstOrDefault?

You should take care of following points while choosing Single, SingleOrDefault, First and FirstOrDefault.

1. When you want an exception to be thrown if the result set contains many records, use **Single** or **SingleOrDefault**.
2. When you want a default value is returned if the result set contains no record, use **SingleOrDefault**.
3. When you always want one record no matter what the result set contains, use **First** or **FirstOrDefault**.
4. When you want a default value if the result set contains no record, use **FirstOrDefault**.

Which one is fast between SingleOrDefault and FirstOrDefault?

FirstOrDefault usually perform faster as compared **SingleOrDefault**, since these iterate the collection until they find the first match. While **SingleOrDefault** iterate the whole collection to find one single match.

What is difference between Select and SelectMany in LINQ?

Select and **SelectMany** are projection operators. **Select** operator is used to select value from a collection and **SelectMany** operator is used to select values from a collection of collection i.e. **nested collection**.

Select operator produce one result value for every source value while **SelectMany** produce a single result that contains a concatenated value for every source value. Actually, **SelectMany** operator flatten **IEnumerable<IEnumerable<T>>** to **IEnumerable<T>** i.e. **list of list to list**.

What is difference among Any, All and Contains?

Any - It checks whether at least one element in a collection satisfy a specified condition. It returns true if at least one element satisfy the condition else returns false if they do not.

All - It checks whether all the elements in a collection satisfy a specified condition. It returns true if all the elements satisfy the condition else returns false if they do not.

Eg-

```
string[] names = { "Rohan", "Raj", "Rahul", "Rajesh", "Rita" };
bool IsFirstLetterR = names.All(name => name.StartsWith("R")); //return true
string[] skills = { "C", "C++", "C#", "ASP.NET", "LINQ" };
bool IsFirstLetterC = skills.Any(s => s.StartsWith("C")); //return true
string[] hobbies = { "Swimming", "Cricket", "Singing", "Watching Movie" };
bool IsHobby = hobbies.Contains("Swimming"); //return true
```

Write LINQ query to find 2nd highest salary?

```
DataContext context = new DataContext();
var q= context.tblEmployee.GroupBy(ord => ord.Salary)
.OrderByDescending(f => f.Key)
.Skip(1) //second, third ..nth highest salary where n=1,2...n-1
.First()
.Select(ord=>ord.Salary)
.Distinct();
```

What is difference between IEnumerable and IQueryable?

IEnumerable	IQueryable
Best to query data from in-memory collections like List, Array etc.	Best to query data from out-memory (like remote database, service) collections.
While querying data from database, IEnumerable execute select query on server side, load data in-memory on client side and then filter data. For Example: DataContext context = new DataContext(); IEnumerable<Employee> list = context.Employees.Where(p => p.Name.StartsWith("S")); list = list.Take<Employee>(10); Generated SQL having no Top Keyword: SELECT [t0].[EmpID], [t0].[EmpName], [t0].[Salary] FROM [Employee] AS [t0] WHERE [t0].[EmpName] LIKE @p0	While querying data from database, IQueryable execute select query on server side with all filters. For Example: DataContext context = new DataContext(); IQueryable<Employee> list = context.Employees.Where(p => p.Name.StartsWith("S")); list = list.Take<Employee>(10); Generated SQL having Top Keyword: SELECT TOP 10 [t0].[EmpID], [t0].[EmpName],[t0].[Salary] FROM [Employee] AS [t0] WHERE [t0].[EmpName] LIKE @p0
Suitable for LINQ to Object and LINQ to XML queries.	Suitable for LINQ to SQL queries.

What is difference between LINQ and Stored Procedures?

There are the following differences between LINQ and Stored Procedures.

1. Stored procedures are faster as compared to LINQ query since they have a predictable execution plan and can take the full advantage of SQL features. Hence, when a stored procedure is being executed next time, the database used the cached execution plan to execute that stored procedure.
2. LINQ has full **type checking** at compile-time and **IntelliSense** support in Visual Studio as compared to stored procedure. This powerful feature helps you to avoid run-time errors.
3. LINQ allows debugging through .NET debugger as compared to stored procedure.
4. LINQ also supports various .NET framework features like multi –threading as compared to stored procedures.
5. LINQ provides the uniform programming model (means common query syntax) to query the multiple databases while you need to re-write the stored procedure for different databases.
6. **Stored procedure is a best way for writing complex queries as compared to LINQ.**
7. Deploying LINQ based application is much easy and simple as compared to stored procedures based. Since in case of stored procedures, you need to provide a SQL script for deployment but in case of LINQ everything gets complied into the DLLs. Hence you need to deploy only DLLs.

What are disadvantages of LINQ over Stored Procedures?

Ans. There are the following disadvantages of LINQ over Stored Procedures.

1. LINQ query is compiled each and every time while stored procedures re-used the cached execution plan to execute. Hence, LINQ query takes more time in execution as compared to stored procedures.
2. **LINQ is not the good for writing complex queries as compared to stored procedures.**
3. **LINQ is not a good way for bulk insert and update operations.**
4. **Performance is degraded if you don't write the LINQ query correctly.**
5. **If you have done some changes in your query, you have to recompile it and redeploy its dll to the server.**